CACS355 · BCA VI Semester · Network Programming

Unit 2: Internet Addresses

Theory, exam programs and interview notes, in **Java** (for the exam) and **Go** (for jobs), side by side.

Syllabus topic	Theory	Java	Go
2.1 The InetAddress class: creating new InetAddress objects, getter methods	A1, A2	B1-B3	C1-C3
2.2 Methods: address types, testing reachability, object methods	A3, A4	B4-B6	C4-C6
2.3 Inet4Address and Inet6Address	A1	B7	C7
2.4 The NetworkInterface class: factory and getter methods	A5	B8	C8
2.5 Useful programs: SpamCheck, processing web server log files	A6, A7	B9, B10	C9, C10

How to read this guide

Part A, Theory is shared by both languages: IP addresses, DNS, address types, interfaces, DNSBL and log files.

Part B, Java is the exam track: every InetAddress/NetworkInterface method in the syllabus, with 10 programs.

Part C, Go is the job track: the same 10 programs in Go, then interview notes.

Part D lists likely exam questions; **Part E** is the Java vs Go cheat sheet.

Read B and C in parallel: B2 then C2, B3 then C3, and so on. The section numbers match. Programs are in `java` and `go` next to this PDF; all were compiled and run, and the outputs shown are real (personal addresses replaced with documentation examples).

Unit 2 has 4 teaching hours and is very program-heavy: almost every subtopic is a short program. Expect at least one "write a program to..." question from here.

Contents

Part A: Theory (common to Java and Go) 4

A1. IP addresses: IPv4 and IPv6 . . . 4
A2. DNS: names and addresses . . . 4
A3. Address types . . . 5
A4. Testing reachability . . . 5
A5. Network interfaces . . . 6
A6. SpamCheck and DNS blackhole lists . . . 6
A7. Processing web server log files . . . 6

Part B: Java (exam track) 8

B1. The InetAddress class . . . 8
B2. Creating new InetAddress objects (2.1) . . . 8
B3. Getter methods (2.1) . . . 9
B4. Address types (2.2) . . . 10
B5. Testing reachability (2.2) . . . 11
B6. Object methods (2.2) . . . 12
B7. Inet4Address and Inet6Address (2.3) . . . 13
B8. The NetworkInterface class (2.4) . . . 14
B9. SpamCheck (2.5) . . . 15
B10. Processing web server log files (2.5) . . . 16
B11. Tips for Unit 2 in the exam . . . 18

Part C: Go (job track) 19

C1. Addresses in Go: net.IP and netip.Addr . . . 19
C2. Creating addresses (Java B2) . . . 19
C3. Getting information (Java B3) . . . 20
C4. Address types (Java B4) . . . 21
C5. Testing reachability (Java B5) . . . 22
C6. Comparing and printing (Java B6) . . . 23
C7. IPv4 vs IPv6 (Java B7) . . . 24
C8. Network interfaces (Java B8) . . . 25
C9. SpamCheck (Java B9) . . . 26
C10. Processing log files (Java B10) . . . 27
C11. Interview notes (Go) . . . 29

Part D: Likely exam questions for Unit 2 30

Short questions . . . 30
Programs (long questions) . . . 30

Part E: Java vs Go cheat sheet (Unit 2) **31**

Part A: Theory (common to Java and Go)

A1. IP addresses: IPv4 and IPv6

Every device on the Internet is identified by an **IP address**. Programs normally use **host names** like www.google.com, and the **DNS** converts them to IP addresses. Java represents an IP address with the `InetAddress` class; Go uses `net.IP` (and the newer `netip.Addr`).

Basis	IPv4	IPv6
Size	32 bits (4 bytes)	128 bits (16 bytes)
Number of addresses	About 4.3 billion (2^{32})	About 3.4 x 10^{38} (2^{128})
Notation	Dotted quad: four decimal numbers 0-255, e.g. 192.168.1.10	Eight groups of four hex digits separated by colons, e.g. 2001:0db8:0000:0000:0000:0000:0000:0001
Shortening	None	Drop leading zeros in a group; replace ONE run of zero groups with ::, so the example becomes 2001:db8::1
Loopback	127.0.0.1	::1
Wildcard (any)	0.0.0.0	::
Java class	Inet4Address	Inet6Address
Go check	ip.To4() != nil	ip.To4() == nil

Mixed notation: an IPv4 address can be written inside IPv6. The **IPv4-mapped** form `::ffff:192.168.1.1` is how an IPv6 socket represents an IPv4 client. The old **IPv4-compatible** form `::192.168.1.1` is deprecated.

A2. DNS: names and addresses

- **Forward lookup:** host name to IP address (A record for IPv4, AAAA record for IPv6). Java: `getByName`; Go: `net.LookupIP`.
- **Reverse lookup:** IP address to host name, using a PTR record in the special domain in-addr.arpa (IPv4) or ip6.arpa (IPv6). Java: `getHostName`; Go: `net.LookupAddr`. Many IPs have no PTR record, so reverse lookups often fail.
- **One name, many addresses:** big sites return several IPs for load balancing and redundancy, and both IPv4 and IPv6 addresses. That is why Java has `getAllByName`.
- **Many names, one address:** one server can host many sites (virtual hosting). A name may be an alias (CNAME) for a **canonical name**. Java: `getCanonicalHostName`; Go: `net.LookupCNAME`.
- **Lookups are slow:** each one is a network round trip, sometimes seconds. Results are therefore **cached**, and programs that do many lookups (A7) run them **in parallel**.

Caching of DNS results in Java

The JVM caches successful lookups (by default 30 seconds) and failed lookups (10 seconds) so a slow DNS server is not asked again and again. The times are set with the security properties `networkaddress.cache.ttl` and `networkaddress.cache.negative.ttl` (-1 means cache forever, 0 means never cache). Go's standard resolver does not cache; the operating system usually does.

A3. Address types

Some address ranges have special meaning. Both Java and Go have methods to test them; this table is worth memorising.

Type	Meaning	IPv4 range	IPv6 range	Java method	Go method
Wildcard	"Any local address": bind to every interface	0.0.0.0	::	isAnyLocalAddress()	IsUnspecified()
Loopback	This machine; never leaves the host	127.0.0.0/8	::1	isLoopbackAddress()	IsLoopback()
Link-local	Auto-assigned; valid only on the local link	169.254.0.0/16	fe80::/10	isLinkLocalAddress()	IsLinkLocalUnicast()
Site-local / private	Usable inside an organisation, not routed on the Internet	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	fec0::/10 (old), fc00::/7	isSiteLocalAddress()	IsPrivate()
Multicast	One sender, a group of receivers (Unit 11)	224.0.0.0/4	ff00::/8	isMulticastAddress()	IsMulticast()
Global unicast	Normal public address	everything else	2000::/3	(none of the above)	IsGlobalUnicast()

Multicast scopes

A multicast address also has a **scope**: how far its packets may travel. Java has a method for each scope.

Scope	Reaches	IPv4 example	IPv6 prefix	Java method
Node / interface-local	Only this interface	-	ff01::	isMCNodeLocal()
Link-local (subnet)	The local subnet only	224.0.0.0 - 224.0.0.255	ff02::	isMCLinkLocal()
Site-local	The site (e.g. a campus)	239.255.0.0/16	ff05::	isMCSiteLocal()
Organisation-local	Several sites of one organisation	239.192.0.0/14	ff08::	isMCOrgLocal()
Global	The whole Internet	224.0.1.0 - 238.255.255.255	ff0e::	isMCGlobal()

A4. Testing reachability

To check whether a host is "up", programs traditionally send an **ICMP echo request** (what `ping` does) and wait for the echo reply.

- Sending raw ICMP packets needs administrator/root privileges on most systems.
- Without privileges, Java's `isReachable` falls back to opening a TCP connection to port 7 (the echo service).
- Many hosts and firewalls block ICMP and port 7, so **"not reachable" does not prove the host is down**. It only means it did not answer this kind of probe within the timeout.
- A practical alternative, used in the Go version, is to try a TCP connection to a port the service really uses (80, 443).

A5. Network interfaces

A **network interface** is the point where a computer connects to a network. It can be **physical** (Ethernet card, Wi-Fi adapter) or **virtual** (the loopback interface, a VPN adapter, Docker's bridge). One machine usually has several, and one interface can have several IP addresses (IPv4, IPv6 global, IPv6 link-local).

Property	Meaning	Example
Name / display name	System name and human-readable name	lo, wlan1 / "Intel(R) Wireless-AC 9462"
Index	Number the OS gives the interface	1, 11
Addresses	IP addresses assigned to it	192.168.1.74, fe80::...%wlan1
Hardware (MAC) address	48-bit physical address of the adapter	94-E2-3C-...
MTU	Largest packet the interface can send	1500 bytes for Ethernet/Wi-Fi
Flags	Up, loopback, point-to-point, supports multicast, virtual	up=true, loopback=false

Why programs need this: choose which interface a server binds to, find the machine's real LAN address instead of 127.0.0.1, pick the interface for multicast (Unit 11), or read the MAC address for licensing and inventory tools. The commands `ipconfig /all` (Windows) and `ip addr` (Linux) show the same information.

A6. SpamCheck and DNS blackhole lists

A **DNS blackhole list (DNSBL)** is a list of IP addresses known to send spam, published through DNS. Mail servers query it before accepting a message. Using DNS means no special protocol or library is needed: any program that can do a DNS lookup can use it.

Algorithm

- Take the IPv4 address to check, e.g. 207.34.56.23.
- Reverse the four octets: 23.56.34.207.
- Append the DNSBL domain: `23.56.34.207.sbl.spamhaus.org`.
- Look the name up. If it **resolves** (usually to 127.0.0.2 - 127.0.0.11), the IP is **listed as a spammer**. If the lookup fails (unknown host), the IP is **not listed**.
- Every DNSBL keeps 127.0.0.2 permanently listed as a test entry, which is why the programs check it.

> **Real-world caveat**
>
> Spamhaus may refuse queries that arrive through large public DNS resolvers (Google 8.8.8.8, Cloudflare 1.1.1.1) and answer with an error code such as 127.255.255.254 instead. The textbook program only checks *whether* the name resolves, so behind such a resolver it would report every IP as a spammer. A production tool checks the returned address. For the exam, write the textbook version; mention this if asked about limitations.

A7. Processing web server log files

Web servers record every request in an access log. The usual format is the **Common Log Format**:

```
8.8.8.8 - - [01/Nov/2026:10:15:32 +0545] "GET /index.html HTTP/1.1" 200 5120
```

Field	Value in the example	Meaning
host	8.8.8.8	Client IP address (servers log IPs, not names, because lookups are slow)
ident	-	Identity from identd, almost always -
authuser	-	User name if the page needed a login
[date]	[01/Nov/2026:10:15:32 +0545]	Time of the request
"request"	"GET /index.html HTTP/1.1"	Method, path and protocol
status	200	HTTP status code
bytes	5120	Size of the response body

The task: to see who visited, replace each IP with its host name using a reverse DNS lookup. **The problem:** each lookup can take seconds and a log has millions of lines, so a one-at-a-time program is very slow. **The solution:** do lookups in parallel with a thread pool (Java) or goroutines (Go), while keeping the output in the original order.

Part B: Java (exam track)

B1. The InetAddress class

`java.net.InetAddress` represents an IP address, and usually the host name that goes with it. Nearly every other networking class (Socket, ServerSocket, URL, DatagramPacket) uses it.

- **No public constructor.** You create objects with **static factory methods** such as `getByName()`.
- **Immutable:** there are getters but no setters, so an InetAddress is thread-safe and can be shared freely.
- **Two subclasses:** `Inet4Address` and `Inet6Address`. The factory methods return the right one automatically.
- **Lookups are cached** (see A2), and a failed lookup throws the checked exception `UnknownHostException`.
- Import it with `import java.net.*;`.

B2. Creating new InetAddress objects (2.1)

Factory method	What it does	DNS lookup?
`getByName(String host)`	Returns one address for a name, or parses an IP string	Yes for a name; no for an IP string
`getAllByName(String host)`	Returns an array of every address for the host	Yes
`getLocalHost()`	Address of the machine running the program	Asks the OS for the host name
`getLoopbackAddress()`	127.0.0.1 (or ::1). Never throws	No
`getByAddress(byte[] addr)`	Builds an address from 4 or 16 raw bytes	No
`getByAddress(String host, byte[] addr)`	Same, with a host name you choose	No (the name is not checked)

`InetAddressFactory.java`

```
import java.net.*;

// 2.1 Creating InetAddress objects with the static factory methods.
public class InetAddressFactory {
    public static void main(String[] args) {
        String host = args.length > 0 ? args[0] : "www.google.com";
        try {
            // 1. by host name: does a DNS lookup, returns the first address
            InetAddress address = InetAddress.getByName(host);
            System.out.println("getByName          : " + address);

            // 2. every address registered for the host
            InetAddress[] all = InetAddress.getAllByName(host);
            for (InetAddress a : all) {
                System.out.println("getAllByName       : " + a);
            }

            // 3. this machine
            InetAddress me = InetAddress.getLocalHost();
            System.out.println("getLocalHost       : " + me);

            // 4. loopback (127.0.0.1 or ::1), never fails
            System.out.println("getLoopbackAddress : " + InetAddress.getLoopbackAddress());

            // 5. from raw bytes: no DNS lookup at all
            byte[] ip = {(byte) 192, (byte) 168, 1, 1};
```

```
            InetAddress raw = InetAddress.getByAddress(ip);
            System.out.println("getByAddress       : " + raw);
            InetAddress named = InetAddress.getByAddress("myrouter", ip);
            System.out.println("getByAddress(name) : " + named);
        } catch (UnknownHostException e) {
            System.out.println("Could not find host: " + e.getMessage());
        }
    }
}
```

Sample output

```
getByName          : www.google.com/142.251.154.119
getAllByName       : www.google.com/142.251.154.119
getAllByName       : www.google.com/142.251.155.119
   ... (8 IPv4 and 8 IPv6 addresses in total)
getAllByName       : www.google.com/2001:4860:4827:7700:0:0:0:0
getLocalHost       : DESKTOP-XXXXXXX/192.168.1.74
getLoopbackAddress : localhost/127.0.0.1
getByAddress       : /192.168.1.1
getByAddress(name) : myrouter/192.168.1.1
```

- Printing an InetAddress calls `toString()`, which gives **hostname/IP**. When no name is known the part before the slash is empty, as in `/192.168.1.1`.
- Bytes above 127 need a cast: `(byte) 192`, because Java's `byte` is signed (-128 to 127).
- Run with an argument to look up another host: `java InetAddressFactory www.tu.edu.np`.

B3. Getter methods (2.1)

Getter	Returns	Example
`String getHostName()`	Host name. If the object was made from an IP, does a reverse lookup; if that fails, returns the IP string	dns.google
`String getCanonicalHostName()`	The real (canonical) name; always asks DNS	edge-star-mini-shv-03-bom2.facebook.com
`String getHostAddress()`	IP address as text	57.144.124.1
`byte[] getAddress()`	IP address as raw bytes (4 for IPv4, 16 for IPv6), in network byte order	{57, -112, 124, 1}

`InetAddressGetters.java`

```
import java.net.*;

// 2.1 Getter methods: getHostName, getCanonicalHostName, getHostAddress, getAddress.
public class InetAddressGetters {
    public static void main(String[] args) {
        String host = args.length > 0 ? args[0] : "www.facebook.com";
        try {
            InetAddress address = InetAddress.getByName(host);
            System.out.println("Host name      : " + address.getHostName());
            System.out.println("Canonical name : " + address.getCanonicalHostName());
            System.out.println("IP address     : " + address.getHostAddress());

            byte[] bytes = address.getAddress();
            System.out.print("Raw bytes      : ");
            for (byte b : bytes) {
                int unsigned = b < 0 ? b + 256 : b;   // Java bytes are signed (-128..127)
                System.out.print(unsigned + " ");
            }
            System.out.println();
            System.out.println("IP version     : " + (bytes.length == 4 ? "IPv4" : "IPv6"));
```

```
            // reverse lookup: IP address -> host name
            InetAddress dns = InetAddress.getByName("8.8.8.8");
            System.out.println("8.8.8.8 is     : " + dns.getHostName());
        } catch (UnknownHostException e) {
            System.out.println("Could not find host: " + e.getMessage());
        }
    }
}
```

Sample output

```
Host name      : www.facebook.com
Canonical name : edge-star-mini-shv-03-bom2.facebook.com
IP address     : 57.144.124.1
Raw bytes      : 57 144 124 1
IP version     : IPv4
8.8.8.8 is     : dns.google
```

- **Signed bytes:** 144 is stored as -112. Convert with `b < 0 ? b + 256 : b` or `b & 0xFF`. This is a favourite exam detail.
- Checking `getAddress().length` is the simplest way to tell IPv4 (4) from IPv6 (16).
- There are **no setter methods**: you cannot change an InetAddress, only create a new one.

B4. Address types (2.2)

Method	True when the address is…
`isAnyLocalAddress()`	the wildcard 0.0.0.0 or ::
`isLoopbackAddress()`	loopback, 127.x.x.x or ::1
`isLinkLocalAddress()`	IPv6 link-local fe80::/10 (also 169.254.x.x for IPv4)
`isSiteLocalAddress()`	private: 10.x, 172.16-31.x, 192.168.x, or IPv6 fec0::/10
`isMulticastAddress()`	multicast: 224.0.0.0 - 239.255.255.255 or ff00::/8
`isMCGlobal()`	global-scope multicast
`isMCOrgLocal()`	organisation-wide multicast
`isMCSiteLocal()`	site-wide multicast
`isMCLinkLocal()`	subnet-wide multicast
`isMCNodeLocal()`	interface-local multicast

`AddressTypes.java`

```
import java.net.*;

// 2.2 Address types: classify an address with the isXXX() methods.
public class AddressTypes {
    public static void main(String[] args) {
        String[] hosts = args.length > 0 ? args : new String[] {
            "0.0.0.0", "127.0.0.1", "169.254.10.1", "192.168.1.10", "8.8.8.8",
            "224.0.0.1", "239.255.255.250", "::1", "fe80::1", "ff0e::1"
        };
        for (String host : hosts) {
            try {
                InetAddress address = InetAddress.getByName(host);
                System.out.println(host + " -> " + type(address));
            } catch (UnknownHostException e) {
                System.out.println(host + " -> unknown host");
            }
        }
    }
```

```
    static String type(InetAddress a) {
        if (a.isAnyLocalAddress()) return "wildcard (any local) address";
        if (a.isLoopbackAddress()) return "loopback address";
        if (a.isLinkLocalAddress()) return "link-local address";
        if (a.isSiteLocalAddress()) return "site-local (private) address";
        if (a.isMulticastAddress()) {
            if (a.isMCGlobal()) return "global multicast address";
            if (a.isMCOrgLocal()) return "organization-wide multicast address";
            if (a.isMCSiteLocal()) return "site-wide multicast address";
            if (a.isMCLinkLocal()) return "subnet-wide (link-local) multicast address";
            if (a.isMCNodeLocal()) return "interface-local (node-local) multicast address";
            return "multicast address of unknown scope";
        }
        return "global (public) unicast address";
    }
}
```

Sample output

```
0.0.0.0 -> wildcard (any local) address
127.0.0.1 -> loopback address
169.254.10.1 -> link-local address
192.168.1.10 -> site-local (private) address
8.8.8.8 -> global (public) unicast address
224.0.0.1 -> subnet-wide (link-local) multicast address
239.255.255.250 -> site-wide multicast address
::1 -> loopback address
fe80::1 -> link-local address
ff0e::1 -> global multicast address
```

The order of the if statements matters: the multicast scope methods only make sense after `isMulticastAddress()` is true. The textbook calls this program **IPCharacteristics**.

B5. Testing reachability (2.2)

Method	Meaning
`boolean isReachable(int timeout)`	Tries to reach the host within timeout milliseconds
`boolean isReachable(NetworkInterface ni, int ttl, int timeout)`	Same, but sends from a specific interface and limits the number of hops (ttl)

Reachability.java

```
import java.io.*;
import java.net.*;

// 2.2 Testing reachability with isReachable(timeout).
public class Reachability {
    public static void main(String[] args) {
        String[] hosts = args.length > 0 ? args
                : new String[] {"localhost", "www.google.com", "10.255.255.1"};
        for (String host : hosts) {
            try {
                InetAddress address = InetAddress.getByName(host);
                boolean ok = address.isReachable(3000);   // wait at most 3000 ms
                System.out.println(host + (ok ? " is reachable" : " is NOT reachable"));
            } catch (UnknownHostException e) {
                System.out.println(host + " -> unknown host");
            } catch (IOException e) {
                System.out.println(host + " -> network error: " + e.getMessage());
            }
        }
    }
}
```

Sample output

```
localhost is reachable
www.google.com is reachable
10.255.255.1 is NOT reachable
```

- It uses ICMP echo (ping) when the program has privileges, otherwise a TCP connection to port 7. See A4.
- It throws `IOException` on a network error, so catch that as well as UnknownHostException.
- 10.255.255.1 is a private address nobody uses, so the request times out after 3 seconds.

B6. Object methods (2.2)

InetAddress overrides the three methods inherited from `java.lang.Object`:

Method	Behaviour
`boolean equals(Object o)`	True if o is an InetAddress with the **same IP address**. The host name is ignored.
`int hashCode()`	Computed from the IP address only, so equal objects have equal hash codes (safe to use in HashMap/HashSet).
`String toString()`	"hostname/dotted-address", e.g. www.google.com/142.251.154.119. Name part is empty if unknown.

`ObjectMethods.java`

```
import java.net.*;

// 2.2 Object methods: equals(), hashCode() and toString() of InetAddress.
public class ObjectMethods {
    public static void main(String[] args) {
        try {
            InetAddress a = InetAddress.getByName("127.0.0.1");
            InetAddress b = InetAddress.getByAddress("loopback", new byte[] {127, 0, 0, 1});
            InetAddress c = InetAddress.getByName("127.0.0.2");

            // toString() = "hostname/ip"; hostname is empty if not known yet
            System.out.println("a = " + a);
            System.out.println("b = " + b);
            System.out.println("c = " + c);

            // equals() compares ONLY the IP address, not the host name
            System.out.println("a.equals(b) : " + a.equals(b));
            System.out.println("a.equals(c) : " + a.equals(c));

            // hashCode() is computed from the IP address only
            System.out.println("a.hashCode(): " + a.hashCode());
            System.out.println("b.hashCode(): " + b.hashCode());
        } catch (UnknownHostException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Sample output

```
a = /127.0.0.1
b = loopback/127.0.0.1
c = /127.0.0.2
a.equals(b) : true
a.equals(c) : false
a.hashCode(): 2130706433
b.hashCode(): 2130706433
```

2130706433 is simply 127.0.0.1 read as one 32-bit number (127 x 2^{24} + 1). a and b have different names but the same IP, so they are equal.

B7. Inet4Address and Inet6Address (2.3)

Both are subclasses of InetAddress. You rarely create them directly: the factory methods return the right one, and you test which with `instanceof`.

Basis	Inet4Address	Inet6Address
Address size	4 bytes	16 bytes
`getHostAddress()` format	192.168.1.1	2001:4860:4860:0:0:0:0:8888 (Java does not shorten with ::)
Extra methods	None beyond InetAddress	`isIPv4CompatibleAddress()`, `getScopeId()`, `getScopedInterface()`
Scope	-	Link-local addresses need a scope (interface), written fe80::1%wlan1

`IPVersion.java`

```
import java.net.*;

// 2.3 Inet4Address and Inet6Address: find out which version an address is.
public class IPVersion {
    public static void main(String[] args) {
        String[] hosts = args.length > 0 ? args : new String[] {
            "127.0.0.1", "::1", "2001:4860:4860::8888", "::ffff:192.168.1.1", "www.google.com"
        };
        for (String host : hosts) {
            try {
                InetAddress address = InetAddress.getByName(host);
                if (address instanceof Inet4Address) {
                    System.out.println(host + " -> IPv4 (" + address.getHostAddress() + ")");
                } else if (address instanceof Inet6Address) {
                    Inet6Address v6 = (Inet6Address) address;
                    System.out.println(host + " -> IPv6 (" + v6.getHostAddress() + ")"
                            + (v6.isIPv4CompatibleAddress() ? " IPv4-compatible" : ""));
                }
                // second method: count the bytes (4 = IPv4, 16 = IPv6)
                System.out.println("    getAddress().length = " + address.getAddress().length);
            } catch (UnknownHostException e) {
                System.out.println(host + " -> unknown host");
            }
        }
    }
}
```

Sample output

```
127.0.0.1 -> IPv4 (127.0.0.1)
    getAddress().length = 4
::1 -> IPv6 (0:0:0:0:0:0:0:1) IPv4-compatible
    getAddress().length = 16
2001:4860:4860::8888 -> IPv6 (2001:4860:4860:0:0:0:0:8888)
    getAddress().length = 16
::ffff:192.168.1.1 -> IPv4 (192.168.1.1)
    getAddress().length = 4
www.google.com -> IPv4 (142.251.154.119)
    getAddress().length = 4
```

- **IPv4-mapped addresses become Inet4Address:** Java converts `::ffff:192.168.1.1` to a plain 4-byte IPv4 address.

- **::1 shows "IPv4-compatible":** `isIPv4CompatibleAddress()` only checks that the first 12 bytes are zero, which is also true for ::1. A useful example of reading what a method actually tests.

B8. The NetworkInterface class (2.4)

`java.net.NetworkInterface` represents a local interface (A5). Like InetAddress it has no public constructor.

Factory methods (static)

Method	Returns
`getByName(String name)`	The interface with that name ("lo", "eth0", "wlan1"), or **null** if none
`getByInetAddress(InetAddress a)`	The interface that has that IP address, or null
`getByIndex(int index)`	The interface with that index, or null (Java 7+)
`getNetworkInterfaces()`	An `Enumeration<NetworkInterface>` of every interface

Getter methods

Method	Returns
`getName()`	Short name, e.g. wlan1
`getDisplayName()`	Human-readable name, e.g. Intel(R) Wireless-AC 9462
`getInetAddresses()`	`Enumeration<InetAddress>` of the interface's addresses
`getInterfaceAddresses()`	List of InterfaceAddress (address + prefix length + broadcast)
`getHardwareAddress()`	MAC address as byte[] (null for loopback)
`getMTU()`	Maximum transmission unit
`isUp(), isLoopback(), isVirtual(), isPointToPoint(), supportsMulticast()`	Interface flags
`getParent(), getSubInterfaces()`	Virtual sub-interface relationships

`InterfaceLister.java`

```
import java.net.*;
import java.util.*;

// 2.4 NetworkInterface: factory methods and getter methods.
public class InterfaceLister {
    public static void main(String[] args) {
        try {
            // factory method 1: every interface on this machine
            Enumeration<NetworkInterface> interfaces = NetworkInterface.getNetworkInterfaces();
            while (interfaces.hasMoreElements()) {
                NetworkInterface ni = interfaces.nextElement();
                if (!ni.isUp()) continue;                // skip disabled interfaces
                System.out.println("Name: " + ni.getName() + "  (" + ni.getDisplayName() + ")");
                System.out.println("    loopback=" + ni.isLoopback() + "  MTU=" + ni.getMTU()
                        + "  MAC=" + mac(ni.getHardwareAddress()));
                Enumeration<InetAddress> addresses = ni.getInetAddresses();
                while (addresses.hasMoreElements()) {
                    System.out.println("    address: " + addresses.nextElement().getHostAddress());
                }
            }

            // factory method 2: by name
            NetworkInterface lo = NetworkInterface.getByName("lo");
            System.out.println("getByName(\"lo\")  -> "
```

```
                    + (lo == null ? "no such interface" : lo.getDisplayName()));

            // factory method 3: by IP address
            InetAddress local = InetAddress.getByName("127.0.0.1");
            NetworkInterface byAddress = NetworkInterface.getByInetAddress(local);
            System.out.println("getByInetAddress(127.0.0.1) -> "
                    + (byAddress == null ? "none" : byAddress.getName()));
        } catch (SocketException e) {
            System.out.println("Could not list interfaces: " + e.getMessage());
        } catch (UnknownHostException e) {
            System.out.println("Bad address: " + e.getMessage());
        }
    }

    // format a MAC address as 00-1A-2B-3C-4D-5E
    static String mac(byte[] bytes) {
        if (bytes == null) return "none";
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < bytes.length; i++) {
            sb.append(String.format("%02X", bytes[i]));
            if (i < bytes.length - 1) sb.append("-");
        }
        return sb.toString();
    }
}
```

Sample output

```
Name: lo  (Software Loopback Interface 1)
    loopback=true  MTU=-1  MAC=none
    address: 127.0.0.1
    address: 0:0:0:0:0:0:0:1
Name: wlan1  (Intel(R) Wireless-AC 9462)
    loopback=false  MTU=1500  MAC=AA-BB-CC-DD-EE-FF
    address: 192.168.1.74
    address: 2001:db8:62:f34d:933d:f993:682:35ea
    address: fe80:0:0:0:dbe8:f092:b3df:1d2a%wlan1
getByName("lo")  -> Software Loopback Interface 1
getByInetAddress(127.0.0.1) -> lo
```

- Most methods throw the checked `SocketException`.
- Loop over an Enumeration with `hasMoreElements()` / `nextElement()` (the exam-safe way).
- Java gives Windows interfaces Unix-style names (lo, wlan1, eth0), so `getByName("lo")` works on every OS. Go uses the real Windows names (C8).
- The `%wlan1` after the fe80 address is the scope ID: link-local addresses are only meaningful on one interface.

B9. SpamCheck (2.5)

Implements the DNSBL algorithm from A6: reverse the octets, add `sbl.spamhaus.org`, and treat a successful lookup as "listed".

`SpamCheck.java`

```
import java.net.*;

// 2.5 SpamCheck: ask a DNS blackhole list (DNSBL) whether an IP is a known spammer.
public class SpamCheck {
    public static final String BLACKHOLE = "sbl.spamhaus.org";

    public static void main(String[] args) {
        String[] ips = args.length > 0 ? args : new String[] {"127.0.0.2", "8.8.8.8"};
        for (String ip : ips) {
            if (isSpammer(ip)) {
                System.out.println(ip + " is a known spammer.");
```

```
            } else {
                System.out.println(ip + " appears legitimate.");
            }
        }
    }

    private static boolean isSpammer(String ip) {
        try {
            InetAddress address = InetAddress.getByName(ip);
            byte[] quad = address.getAddress();
            String query = BLACKHOLE;
            for (byte octet : quad) {                    // reverse the octets:
                int unsignedByte = octet < 0 ? octet + 256 : octet;
                query = unsignedByte + "." + query;      // 1.2.3.4 -> 4.3.2.1.sbl.spamhaus.org
            }
            InetAddress.getByName(query);                // resolves -> listed
            return true;
        } catch (UnknownHostException e) {               // does not resolve -> not listed
            return false;
        }
    }
}
```

Sample output

```
127.0.0.2 is a known spammer.
8.8.8.8 appears legitimate.
```

- The key trick is that `UnknownHostException` is not an error here, it is the "not a spammer" answer.
- Building the query with `query = octet + "." + query` reverses the order automatically, because each new octet is added in front.
- Check your own IPs with `java SpamCheck 1.2.3.4 5.6.7.8`.

B10. Processing web server log files (2.5)

Simple version: one lookup at a time

Weblog.java

```
import java.io.*;
import java.net.*;

// 2.5 Processing web server log files: replace each IP address with its host name.
public class Weblog {
    public static void main(String[] args) {
        String file = args.length > 0 ? args[0] : "access.log";
        try (BufferedReader in = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = in.readLine()) != null) {
                int index = line.indexOf(' ');           // IP is the first field
                String ip = line.substring(0, index);
                String rest = line.substring(index);
                try {
                    InetAddress address = InetAddress.getByName(ip);
                    System.out.println(address.getHostName() + rest);  // reverse DNS lookup
                } catch (UnknownHostException e) {
                    System.out.println(line);            // keep the line unchanged
                }
            }
        } catch (IOException e) {
            System.out.println("Could not read " + file + ": " + e.getMessage());
        }
    }
}
```

A sample log, `access.log`, is in the java folder. Run with `java Weblog` or `java Weblog path/to/log`.

Sample output

```
dns.google - - [01/Nov/2026:10:15:32 +0545] "GET /index.html HTTP/1.1" 200 5120
one.one.one.one - - [01/Nov/2026:10:15:40 +0545] "GET /about.html HTTP/1.1" 200 2310
dns9.quad9.net - - [01/Nov/2026:10:16:02 +0545] "GET /images/logo.png HTTP/1.1" 200 15400
kubernetes.docker.internal - - [01/Nov/2026:10:16:10 +0545] "POST /login HTTP/1.1" 302 0
192.0.2.10 - - [01/Nov/2026:10:16:45 +0545] "GET /missing.html HTTP/1.1" 404 512
dns.google - - [01/Nov/2026:10:17:03 +0545] "GET /index.html HTTP/1.1" 304 0
```

- 192.0.2.10 has no PTR record, so `getHostName()` returns the IP unchanged. (Only a badly formed IP would throw UnknownHostException.)
- 127.0.0.1 became kubernetes.docker.internal because Docker Desktop added that name to the Windows hosts file. On most PCs it shows localhost. Reverse lookups read the hosts file before DNS.

Faster version: thread pool (PooledWeblog)

The textbook improves the program by doing the slow lookups in a pool of threads. Each line becomes a `Callable` task; the `Future` objects are kept in a list in file order, so the output order is preserved.

PooledWeblog.java

```
import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.*;

// 2.5 Faster log processing: DNS lookups run in a thread pool, output keeps file order.
public class PooledWeblog {
    public static void main(String[] args) {
        String file = args.length > 0 ? args[0] : "access.log";
        ExecutorService pool = Executors.newFixedThreadPool(4);
        List<Future<String>> results = new ArrayList<>();

        try (BufferedReader in = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = in.readLine()) != null) {
                results.add(pool.submit(new LookupTask(line)));   // lookups run in parallel
            }
        } catch (IOException e) {
            System.out.println("Could not read " + file + ": " + e.getMessage());
        }

        for (Future<String> result : results) {                // print in original order
            try {
                System.out.println(result.get());              // waits for that lookup
            } catch (InterruptedException | ExecutionException e) {
                System.out.println("Lookup failed: " + e.getMessage());
            }
        }
        pool.shutdown();
    }
}

class LookupTask implements Callable<String> {
    private String line;

    LookupTask(String line) {
        this.line = line;
    }

    public String call() {
        int index = line.indexOf(' ');
        String ip = line.substring(0, index);
        try {
            return InetAddress.getByName(ip).getHostName() + line.substring(index);
```

```
        } catch (UnknownHostException e) {
            return line;
        }
    }
}
```

- `Callable<String>` is like Runnable, but `call()` returns a value.
- `pool.submit(task)` starts the task and immediately returns a `Future`; `future.get()` waits for the result.
- With 4 threads, 4 slow lookups wait at the same time instead of one after another.
- Always call `pool.shutdown()`, or the program will not exit.

B11. Tips for Unit 2 in the exam

- Most questions are "write a program to" find the IP of a host, find the host name of an IP, list all IPs of a host, find the local host's address, check address type, check reachability, list network interfaces, or show the MAC address. Each is 10-20 lines; learn B2-B8 well.
- Always handle `UnknownHostException` (InetAddress) or `SocketException` (NetworkInterface). Use `IOException` for isReachable.
- Remember: InetAddress has **no public constructor** and **no setters**. Both are common short questions.
- For SpamCheck and Weblog, write the algorithm steps first (A6, A7), then the code.
- Show the output after each program.

Part C: Go (job track)

C1. Addresses in Go: net.IP and netip.Addr

Go has two types for IP addresses:

	`net.IP` **(package net)**	`netip.Addr` **(package net/netip, Go 1.18+)**
What it is	A byte slice: `type IP []byte`, 4 or 16 bytes	A small immutable value type
Compare	`ip.Equal(other)`; == does not compile	`a == b` works
Map key	Not allowed (slices can't be keys)	Allowed
Used by	Most of the net package (Dial, LookupIP, Interfaces)	Newer APIs and high-performance code
Java equivalent	InetAddress (without the host name)	InetAddress (immutable, comparable)

Go never stores a host name inside an address. A lookup function returns addresses, and you keep the name yourself. Lookups return an `error` instead of throwing UnknownHostException.

Java (Unit 2)	**Go**
`InetAddress.getByName(host)`	`net.ResolveIPAddr("ip", host)`
`InetAddress.getAllByName(host)`	`net.LookupIP(host)`
`InetAddress.getLocalHost()`	`os.Hostname()` + `net.LookupIP`
`getByAddress(bytes)`	`net.IP{192,168,1,1}` or `net.ParseIP("...")`
`getHostName()` (reverse)	`net.LookupAddr(ip)`
`getCanonicalHostName()`	`net.LookupCNAME(host)`
`getHostAddress()`	`ip.String()`
`getAddress()`	`ip.To4()` / `[]byte(ip)`
`isLoopbackAddress()` etc.	`ip.IsLoopback()` etc.
`isReachable(timeout)`	No equivalent; `net.DialTimeout` (C5)
`equals()` / `toString()`	`ip.Equal()` / `ip.String()`
`Inet4Address / Inet6Address`	`ip.To4() != nil`; `addr.Is4() / Is6()`
`NetworkInterface`	`net.Interface`, `net.Interfaces()`

C2. Creating addresses (Java B2)

`go/factory/main.go`

```
package main

import (
    "fmt"
    "log"
    "net"
    "os"
)

// 2.1 Creating IP addresses: DNS lookups and parsing (Java: InetAddress factory methods).
func main() {
    host := "www.google.com"
```

```
    if len(os.Args) > 1 {
        host = os.Args[1]
    }

    // 1. first address of a host (Java: getByName)
    addr, err := net.ResolveIPAddr("ip", host)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Println("ResolveIPAddr :", addr)

    // 2. every address of the host (Java: getAllByName)
    ips, err := net.LookupIP(host)
    if err != nil {
        log.Fatal(err)
    }
    for _, ip := range ips {
        fmt.Println("LookupIP      :", ip)
    }

    // 3. this machine (Java: getLocalHost)
    name, _ := os.Hostname()
    local, _ := net.LookupIP(name)
    fmt.Println("Hostname      :", name, local)

    // 4. loopback (Java: getLoopbackAddress)
    fmt.Println("Loopback      :", net.IPv4(127, 0, 0, 1), net.IPv6loopback)

    // 5. from text or raw bytes: no DNS lookup (Java: getByAddress)
    fmt.Println("ParseIP       :", net.ParseIP("192.168.1.1"))
    fmt.Println("From bytes    :", net.IP{192, 168, 1, 1})
}
```

Sample output

```
ResolveIPAddr : 142.251.157.119
LookupIP      : 2001:4860:4827:7700::
   ... (8 IPv6 and 8 IPv4 addresses in total)
LookupIP      : 142.251.157.119
Hostname      : DESKTOP-XXXXXXX [fe80::dbe8:f092:b3df:1d2a 2001:db8:62:f34d:... 192.168.1.74]
Loopback      : 127.0.0.1 ::1
ParseIP       : 192.168.1.1
From bytes    : 192.168.1.1
```

- Go prints IPv6 in the short form (`2001:4860:4827:7700::`); Java prints every group (`2001:4860:4827:7700:0:0:0:0`).
- `net.ParseIP` returns `nil` for invalid text. Always check it; Java would throw instead.
- No casts are needed for bytes above 127: Go's `byte` is unsigned.

C3. Getting information (Java B3)

go/getters/main.go

```
package main

import (
    "fmt"
    "log"
    "net"
    "os"
)

// 2.1 Getting information about an address (Java: InetAddress getter methods).
func main() {
    host := "www.facebook.com"
    if len(os.Args) > 1 {
```

```
        host = os.Args[1]
    }

    ips, err := net.LookupIP(host)
    if err != nil {
        log.Fatal(err)
    }
    ip := ips[0]

    cname, _ := net.LookupCNAME(host) // Java: getCanonicalHostName
    fmt.Println("Host name      :", host)
    fmt.Println("Canonical name :", cname)
    fmt.Println("IP address     :", ip.String()) // Java: getHostAddress

    if v4 := ip.To4(); v4 != nil { // Java: getAddress().length == 4
        fmt.Println("Raw bytes      :", []byte(v4)) // Go bytes are unsigned (0..255)
        fmt.Println("IP version     : IPv4")
    } else {
        fmt.Println("Raw bytes      :", []byte(ip))
        fmt.Println("IP version     : IPv6")
    }

    // reverse lookup: IP address -> host names (Java: getHostName)
    names, err := net.LookupAddr("8.8.8.8")
    if err != nil {
        log.Fatal(err)
    }
    fmt.Println("8.8.8.8 is     :", names)
}
```

Sample output

```
Host name      : www.facebook.com
Canonical name : star-mini.c10r.facebook.com.
IP address     : 2a03:2880:f33e:8:face:b00c:0:25de
Raw bytes      : [42 3 40 128 243 62 0 8 250 206 176 12 0 0 37 222]
IP version     : IPv6
8.8.8.8 is     : [dns.google.]
```

- DNS names returned by Go end with a dot (`dns.google.`): this is the fully qualified form. Use `strings.TrimSuffix(name, ".")` to remove it.
- LookupAddr returns a **slice** of names, because an IP can have several PTR records. Java's getHostName returns just one.
- This run got an IPv6 address first while Java got IPv4: the order of addresses depends on the resolver and the OS.

C4. Address types (Java B4)

go/address_types/main.go

```
package main

import (
    "fmt"
    "net"
    "os"
)

// 2.2 Address types: classify an address with the IsXXX() methods.
func main() {
    hosts := []string{
        "0.0.0.0", "127.0.0.1", "169.254.10.1", "192.168.1.10", "8.8.8.8",
        "224.0.0.1", "239.255.255.250", "::1", "fe80::1", "ff0e::1",
    }
    if len(os.Args) > 1 {
```

```
        hosts = os.Args[1:]
    }
    for _, h := range hosts {
        ip := net.ParseIP(h)
        if ip == nil {
            fmt.Println(h, "-> not an IP address")
            continue
        }
        fmt.Println(h, "->", kind(ip))
    }
}

func kind(ip net.IP) string {
    switch {
    case ip.IsUnspecified():
        return "wildcard (unspecified) address"
    case ip.IsLoopback():
        return "loopback address"
    case ip.IsLinkLocalUnicast():
        return "link-local address"
    case ip.IsPrivate():
        return "private (site-local) address"
    case ip.IsInterfaceLocalMulticast():
        return "interface-local multicast address"
    case ip.IsLinkLocalMulticast():
        return "link-local multicast address"
    case ip.IsMulticast():
        return "multicast address (wider scope)"
    case ip.IsGlobalUnicast():
        return "global (public) unicast address"
    }
    return "unknown type"
}
```

Sample output

```
0.0.0.0 -> wildcard (unspecified) address
127.0.0.1 -> loopback address
169.254.10.1 -> link-local address
192.168.1.10 -> private (site-local) address
8.8.8.8 -> global (public) unicast address
224.0.0.1 -> link-local multicast address
239.255.255.250 -> multicast address (wider scope)
::1 -> loopback address
fe80::1 -> link-local address
ff0e::1 -> multicast address (wider scope)
```

- Go's `switch` with no expression is a clean replacement for Java's chain of if statements.
- Go's standard library only names the interface-local and link-local multicast scopes. For site/org/global scope you read the bits yourself (for IPv6, scope = `ip[1] & 0x0f`: 5 site, 8 organisation, e global).
- `IsPrivate()` covers the modern IPv6 private range fc00::/7; Java's isSiteLocalAddress only knows the old fec0::/10.

C5. Testing reachability (Java B5)

go/reachability/main.go

```
package main

import (
    "errors"
    "fmt"
    "net"
    "os"
    "time"
)
```

```
// 2.2 Testing reachability. Go has no isReachable(): ICMP ping needs raw sockets
// (admin rights), so we try a TCP connection with a timeout instead.
func main() {
    hosts := []string{"localhost", "www.google.com", "10.255.255.1"}
    if len(os.Args) > 1 {
        hosts = os.Args[1:]
    }
    for _, h := range hosts {
        fmt.Println(h, "->", check(h, "443", 3*time.Second))
    }
}

func check(host, port string, timeout time.Duration) string {
    conn, err := net.DialTimeout("tcp", net.JoinHostPort(host, port), timeout)
    if err == nil {
        conn.Close()
        return "reachable (port " + port + " open)"
    }
    var netErr net.Error
    if errors.As(err, &netErr) && netErr.Timeout() {
        return "NOT reachable (timed out)"
    }
    // "connection refused" means the host answered, but nothing listens on that port
    return "no service on port " + port + ": " + err.Error()
}
```

Sample output

```
localhost -> no service on port 443: dial tcp [::1]:443: connectex: No connection could be made because the target m
www.google.com -> reachable (port 443 open)
10.255.255.1 -> NOT reachable (timed out)
```

- There are three different results: **connected** (host up, port open), **refused** (host up, port closed) and **timeout** (no answer). Java's isReachable squeezes all of these into one boolean.
- `errors.As(err, &netErr) && netErr.Timeout()` is the standard way to detect a timeout in Go.
- For real ICMP ping in Go, use `golang.org/x/net/icmp` (needs admin rights) or the pro-bing library.

C6. Comparing and printing (Java B6)

go/equality/main.go

```
package main

import (
    "bytes"
    "fmt"
    "net"
    "net/netip"
)

// 2.2 Object methods: comparing and printing addresses (Java: equals, hashCode, toString).
func main() {
    a := net.ParseIP("127.0.0.1") // stored in 16-byte form
    b := net.IP{127, 0, 0, 1}     // stored in 4-byte form
    c := net.ParseIP("127.0.0.2")

    // String() is Go's toString()
    fmt.Println("a =", a.String(), " b =", b.String(), " c =", c.String())

    // net.IP is a byte slice: == does not compile, and comparing bytes is wrong
    fmt.Println("bytes.Equal(a, b):", bytes.Equal(a, b)) // false: 16 bytes vs 4 bytes
    fmt.Println("a.Equal(b)       :", a.Equal(b))        // true: same address
    fmt.Println("a.Equal(c)       :", a.Equal(c))

    // netip.Addr (Go 1.18+) is a value type: == works and it can be a map key
```

```
    x := netip.MustParseAddr("127.0.0.1")
    y := netip.AddrFrom4([4]byte{127, 0, 0, 1})
    fmt.Println("x == y           :", x == y)

    seen := map[netip.Addr]int{} // like using InetAddress in a HashMap (needs hashCode)
    seen[x]++
    seen[y]++
    fmt.Println("map count for x  :", seen[x])
}
```

Sample output

```
a = 127.0.0.1  b = 127.0.0.1  c = 127.0.0.2
bytes.Equal(a, b): false
a.Equal(b)       : true
a.Equal(c)       : false
x == y           : true
map count for x  : 2
```

- **The trap:** `net.ParseIP("127.0.0.1")` stores 16 bytes, while `net.IP{127,0,0,1}` stores 4. Comparing the bytes says "different". Always use `Equal()`, which plays the role of Java's equals().
- `netip.Addr` is comparable with == and works as a map key. That is what Java's equals() + hashCode() pair gives you in a HashMap.

C7. IPv4 vs IPv6 (Java B7)

go/ip_version/main.go

```
package main

import (
    "fmt"
    "net"
    "net/netip"
    "os"
)

// 2.3 IPv4 vs IPv6 (Java: Inet4Address and Inet6Address).
func main() {
    hosts := []string{"127.0.0.1", "::1", "2001:4860:4860::8888", "::ffff:192.168.1.1", "www.google.com"}
    if len(os.Args) > 1 {
        hosts = os.Args[1:]
    }
    for _, h := range hosts {
        ip := net.ParseIP(h)
        if ip == nil { // not a literal address: look up the name
            ips, err := net.LookupIP(h)
            if err != nil {
                fmt.Println(h, "-> unknown host")
                continue
            }
            ip = ips[0]
        }

        // method 1: net.IP.To4() returns nil for a real IPv6 address
        if ip.To4() != nil {
            fmt.Println(h, "-> IPv4", ip)
        } else {
            fmt.Println(h, "-> IPv6", ip)
        }

        // method 2: netip.Addr remembers exactly how the address was written
        if addr, err := netip.ParseAddr(h); err == nil {
            fmt.Printf("    netip: Is4=%v Is6=%v Is4In6=%v\n", addr.Is4(), addr.Is6(), addr.Is4In6())
        }
    }
```

```
}
```

Sample output

```
127.0.0.1 -> IPv4 127.0.0.1
    netip: Is4=true Is6=false Is4In6=false
::1 -> IPv6 ::1
    netip: Is4=false Is6=true Is4In6=false
2001:4860:4860::8888 -> IPv6 2001:4860:4860::8888
    netip: Is4=false Is6=true Is4In6=false
::ffff:192.168.1.1 -> IPv4 192.168.1.1
    netip: Is4=false Is6=true Is4In6=true
www.google.com -> IPv6 2001:4860:4827:7700::
```

- `net.IP.To4()` treats the IPv4-mapped address as IPv4, just like Java turns it into an Inet4Address.
- `netip.Addr` remembers how the address was written: `Is4In6()` is true. Call `addr.Unmap()` to turn it into plain IPv4.
- Unlike Java, Go does not call ::1 "IPv4-compatible".

C8. Network interfaces (Java B8)

go/interfaces/main.go

```
package main

import (
    "fmt"
    "log"
    "net"
)

// 2.4 Network interfaces (Java: NetworkInterface).
func main() {
    // all interfaces (Java: getNetworkInterfaces)
    ifaces, err := net.Interfaces()
    if err != nil {
        log.Fatal(err)
    }
    for _, ifc := range ifaces {
        if ifc.Flags&net.FlagUp == 0 { // skip disabled interfaces
            continue
        }
        fmt.Printf("Name: %s  Index: %d\n", ifc.Name, ifc.Index)
        fmt.Printf("    loopback=%v  MTU=%d  MAC=%v\n",
            ifc.Flags&net.FlagLoopback != 0, ifc.MTU, ifc.HardwareAddr)
        addrs, _ := ifc.Addrs() // Java: getInetAddresses
        for _, a := range addrs {
            fmt.Println("    address:", a) // includes the prefix, e.g. 192.168.1.74/24
        }
    }

    // by name (Java: getByName)
    if lo, err := net.InterfaceByName("lo"); err == nil {
        fmt.Println(`InterfaceByName("lo") ->`, lo.Name)
    } else {
        fmt.Println(`InterfaceByName("lo") ->`, err)
    }

    // by address (Java: getByInetAddress): Go has no direct function, so search
    fmt.Println("interface for 127.0.0.1 ->", byIP(ifaces, net.ParseIP("127.0.0.1")))
}

func byIP(ifaces []net.Interface, ip net.IP) string {
    for _, ifc := range ifaces {
        addrs, _ := ifc.Addrs()
        for _, a := range addrs {
```

```
            if ipnet, ok := a.(*net.IPNet); ok && ipnet.IP.Equal(ip) {
                return ifc.Name
            }
        }
    }
    return "none"
}
```

Sample output

```
Name: Wi-Fi  Index: 11
    loopback=false  MTU=1500  MAC=aa:bb:cc:dd:ee:ff
    address: 2001:db8:62:f34d:933d:f993:682:35ea/64
    address: fe80::dbe8:f092:b3df:1d2a/64
    address: 192.168.1.74/24
Name: Loopback Pseudo-Interface 1  Index: 1
    loopback=true  MTU=-1  MAC=
    address: ::1/128
    address: 127.0.0.1/8
InterfaceByName("lo") -> route ip+net: no such network interface
interface for 127.0.0.1 -> Loopback Pseudo-Interface 1
```

- Go uses the **real OS names**: on Windows "Wi-Fi" and "Loopback Pseudo-Interface 1", so `InterfaceByName("lo")` fails there but works on Linux.
- `Addrs()` returns addresses **with the prefix length** (192.168.1.74/24), which Java only gives through getInterfaceAddresses().
- Flags are a bit set: test them with `ifc.Flags & net.FlagUp != 0`.
- There is no getByInetAddress in Go, so the program loops over the interfaces (`byIP`). Writing small helpers like this is normal Go style.

C9. SpamCheck (Java B9)

go/spamcheck/main.go

```
package main

import (
    "fmt"
    "net"
    "os"
)

// 2.5 SpamCheck: ask a DNS blackhole list (DNSBL) whether an IP is a known spammer.
const blackhole = "sbl.spamhaus.org"

func main() {
    ips := []string{"127.0.0.2", "8.8.8.8"}
    if len(os.Args) > 1 {
        ips = os.Args[1:]
    }
    for _, ip := range ips {
        if isSpammer(ip) {
            fmt.Println(ip, "is a known spammer.")
        } else {
            fmt.Println(ip, "appears legitimate.")
        }
    }
}

func isSpammer(s string) bool {
    ip := net.ParseIP(s).To4()
    if ip == nil {
        return false // this simple check handles IPv4 only
    }
    // reverse the octets: 1.2.3.4 -> 4.3.2.1.sbl.spamhaus.org
```

```
    query := fmt.Sprintf("%d.%d.%d.%d.%s", ip[3], ip[2], ip[1], ip[0], blackhole)
    _, err := net.LookupHost(query)
    return err == nil // resolves -> listed
}
```

Sample output

```
127.0.0.2 is a known spammer.
8.8.8.8 appears legitimate.
```

Go's version is shorter: `ip.To4()` gives unsigned bytes, so `fmt.Sprintf` builds the reversed query in one line. The "error means not listed" logic is the same as Java's catch block.

C10. Processing log files (Java B10)

Simple version

go/weblog/main.go

```
package main

import (
    "bufio"
    "fmt"
    "log"
    "net"
    "os"
    "strings"
)

// 2.5 Processing web server log files: replace each IP address with its host name.
func main() {
    file := "access.log"
    if len(os.Args) > 1 {
        file = os.Args[1]
    }
    f, err := os.Open(file)
    if err != nil {
        log.Fatal(err)
    }
    defer f.Close()

    scanner := bufio.NewScanner(f)
    for scanner.Scan() {
        fmt.Println(resolve(scanner.Text()))
    }
}

func resolve(line string) string {
    ip, rest, found := strings.Cut(line, " ") // IP is the first field
    if !found {
        return line
    }
    names, err := net.LookupAddr(ip) // reverse DNS lookup
    if err != nil || len(names) == 0 {
        return line // keep the line unchanged
    }
    return strings.TrimSuffix(names[0], ".") + " " + rest
}
```

Concurrent version (goroutines + semaphore)

go/weblog_pooled/main.go

```
package main

import (
    "bufio"
    "fmt"
    "log"
    "net"
    "os"
    "strings"
    "sync"
)

// 2.5 Faster log processing: lookups run in goroutines, output keeps file order.
func main() {
    file := "access.log"
    if len(os.Args) > 1 {
        file = os.Args[1]
    }
    f, err := os.Open(file)
    if err != nil {
        log.Fatal(err)
    }
    defer f.Close()

    var lines []string
    scanner := bufio.NewScanner(f)
    for scanner.Scan() {
        lines = append(lines, scanner.Text())
    }

    results := make([]string, len(lines)) // one slot per line keeps the order
    sem := make(chan struct{}, 4)         // at most 4 lookups at once (like a pool of 4)
    var wg sync.WaitGroup
    for i, line := range lines {
        wg.Add(1)
        go func(i int, line string) {
            defer wg.Done()
            sem <- struct{}{}        // take a slot
            defer func() { <-sem }() // give it back
            results[i] = resolve(line)
        }(i, line)
    }
    wg.Wait() // like calling Future.get() on every task

    for _, r := range results {
        fmt.Println(r)
    }
}

func resolve(line string) string {
    ip, rest, found := strings.Cut(line, " ")
    if !found {
        return line
    }
    names, err := net.LookupAddr(ip)
    if err != nil || len(names) == 0 {
        return line
    }
    return strings.TrimSuffix(names[0], ".") + " " + rest
}
```

Sample output (both versions)

```
dns.google - - [01/Nov/2026:10:15:32 +0545] "GET /index.html HTTP/1.1" 200 5120
one.one.one.one - - [01/Nov/2026:10:15:40 +0545] "GET /about.html HTTP/1.1" 200 2310
dns9.quad9.net - - [01/Nov/2026:10:16:02 +0545] "GET /images/logo.png HTTP/1.1" 200 15400
```

```
kubernetes.docker.internal - - [01/Nov/2026:10:16:10 +0545] "POST /login HTTP/1.1" 302 0
192.0.2.10 - - [01/Nov/2026:10:16:45 +0545] "GET /missing.html HTTP/1.1" 404 512
dns.google - - [01/Nov/2026:10:17:03 +0545] "GET /index.html HTTP/1.1" 304 0
```

Java PooledWeblog	Go weblog_pooled
`Executors.newFixedThreadPool(4)`	`sem := make(chan struct{}, 4)` limits to 4 at a time
`pool.submit(new LookupTask(line))`	`go func(i, line) { ... }(i, line)`
`List<Future<String>>` keeps order	`results := make([]string, n)`; each goroutine writes its own index
`future.get()`	`wg.Wait()` (sync.WaitGroup)
`pool.shutdown()`	Nothing: goroutines end when their function returns

`strings.Cut(line, " ")` splits at the first space, like Java's indexOf + substring. Writing to `results[i]` from many goroutines is safe because each goroutine writes a **different** index.

C11. Interview notes (Go)

Q. net.IP or netip.Addr: which should you use?

netip.Addr for new code: it is smaller, immutable, comparable with == and usable as a map key. net.IP is still used by most of the net package; convert with netip.AddrFromSlice and addr.AsSlice().

Q. How do you put a timeout on a DNS lookup?

Use a Resolver with a context: `ctx, cancel := context.WithTimeout(ctx, 2*time.Second); net.DefaultResolver.LookupIPAddr(ctx, host)`. The package-level functions like LookupIP have no timeout of their own.

Q. Why does LookupIP return both IPv4 and IPv6, and which does Dial use?

DNS has separate A and AAAA records and the resolver asks for both. net.Dial tries them using "Happy Eyeballs" (RFC 6555): it races IPv6 and IPv4 and uses whichever connects first.

Q. Which DNS resolver does Go use?

On Linux, a pure-Go resolver that reads /etc/resolv.conf by default, or the C library (cgo) resolver; choose with GODEBUG=netdns=go or netdns=cgo. On Windows and macOS it normally calls the OS. The Go resolver does not cache.

Q. Why can't you trust reverse DNS for security?

Whoever controls the IP block controls its PTR record, so it can claim any name. Do a forward-confirmed lookup: resolve the PTR name and check that it maps back to the same IP.

Q. How do you check whether an IP is inside a network, e.g. for an allowlist?

`_, cidr, _ := net.ParseCIDR("10.0.0.0/8"); cidr.Contains(ip)`, or with netip: `netip.MustParsePrefix("10.0.0.0/8").Contains(addr)`.

Q. How would you get the real client IP in an HTTP server behind a load balancer?

r.RemoteAddr is the load balancer's address. The client IP is in the X-Forwarded-For or X-Real-IP header, but only trust it when the request came from your own proxy.

Q. How do you limit concurrency for many slow lookups?

A buffered channel used as a semaphore (as in weblog_pooled), a fixed number of worker goroutines reading from a jobs channel, or golang.org/x/sync/errgroup with SetLimit.

Part D: Likely exam questions for Unit 2

These are typical of how Unit 2 is asked. Check them against past CACS355 papers and mark the ones that repeat.

Short questions

#	Question	Answer pointer
1	What is the InetAddress class? Why does it have no public constructor?	B1 (factory methods, immutability)
2	List the factory methods of InetAddress with their use.	B2 table
3	Explain the getter methods of InetAddress. Why are there no setters?	B3 table + immutability
4	Differentiate getHostName() and getCanonicalHostName().	B3, A2 (CNAME)
5	Differentiate Inet4Address and Inet6Address.	A1 table + B7 table
6	What is a loopback, wildcard, link-local and site-local address?	A3 table
7	Explain the isReachable() method. Why can it return false for a working host?	B5 + A4
8	How do equals() and hashCode() work for InetAddress?	B6
9	What is the NetworkInterface class? List its factory methods.	A5 + B8 tables
10	What is a DNSBL? How does SpamCheck work?	A6 algorithm
11	Why does InetAddress cache DNS lookups? How is it configured?	A2 caching

Programs (long questions)

#	Write a Java program to...	Program
1	find the IP address of a given host name (e.g. www.google.com)	B2 (getByName)
2	display all IP addresses of a host	B2 (getAllByName)
3	display the host name and IP address of the local machine	B2 (getLocalHost)
4	find the host name of a given IP address	B3 (getHostName)
5	display the host name, canonical name, IP address and raw bytes of a host	B3
6	test the characteristics (type) of an IP address	B4 (AddressTypes)
7	check whether a host is reachable	B5
8	check whether an address is IPv4 or IPv6	B7
9	list all network interfaces with their addresses / find the MAC address	B8
10	check whether an IP address is a known spammer	B9 (SpamCheck)
11	read a web server log file and replace IP addresses with host names (optionally using a thread pool)	B10

> **Answer-writing pattern for program questions**
>
> 1. A one-line statement of the idea ("InetAddress.getByName() performs a DNS lookup..."). 2. The program, with imports and exception handling. 3. Comments on the key method calls. 4. A sample output box. 5. One line on the exception that can occur.

Part E: Java vs Go cheat sheet (Unit 2)

Task	Java	Go
Name to one IP	`InetAddress.getByName(h)`	`net.ResolveIPAddr("ip", h)`
Name to all IPs	`InetAddress.getAllByName(h)`	`net.LookupIP(h)`
IP to name	`getByName(ip).getHostName()`	`net.LookupAddr(ip)`
Canonical name	`a.getCanonicalHostName()`	`net.LookupCNAME(h)`
Local host	`InetAddress.getLocalHost()`	`os.Hostname()`
Loopback	`InetAddress.getLoopbackAddress()`	`net.IPv4(127,0,0,1)`, `net.IPv6loopback`
From bytes / text	`getByAddress(new byte[]{...})`	`net.IP{...}`, `net.ParseIP(s)`
IP as text	`a.getHostAddress()`	`ip.String()`
Raw bytes	`a.getAddress()` (signed!)	`ip.To4()` / `[]byte(ip)` (unsigned)
IPv4 or IPv6?	`a instanceof Inet4Address`	`ip.To4() != nil`
Loopback?	`a.isLoopbackAddress()`	`ip.IsLoopback()`
Private?	`a.isSiteLocalAddress()`	`ip.IsPrivate()`
Multicast?	`a.isMulticastAddress()`	`ip.IsMulticast()`
Reachable?	`a.isReachable(3000)`	`net.DialTimeout("tcp", addr, 3*time.Second)`
Equal?	`a.equals(b)`	`a.Equal(b)` / netip `==`
All interfaces	`NetworkInterface.getNetworkInterfaces()`	`net.Interfaces()`
Interface by name	`NetworkInterface.getByName(n)`	`net.InterfaceByName(n)`
Interface addresses	`ni.getInetAddresses()`	`ifc.Addrs()`
MAC address	`ni.getHardwareAddress()`	`ifc.HardwareAddr`
Lookup failed	`catch (UnknownHostException e)`	`if err != nil`
Parallel lookups	`ExecutorService` + `Future`	goroutines + `WaitGroup` + semaphore

Next: Unit 3, URLs and URIs (URL, URI, URLEncoder/URLDecoder, proxies, GET requests, Authenticator), in the same Java-then-Go layout.